

COS214 Practical 5

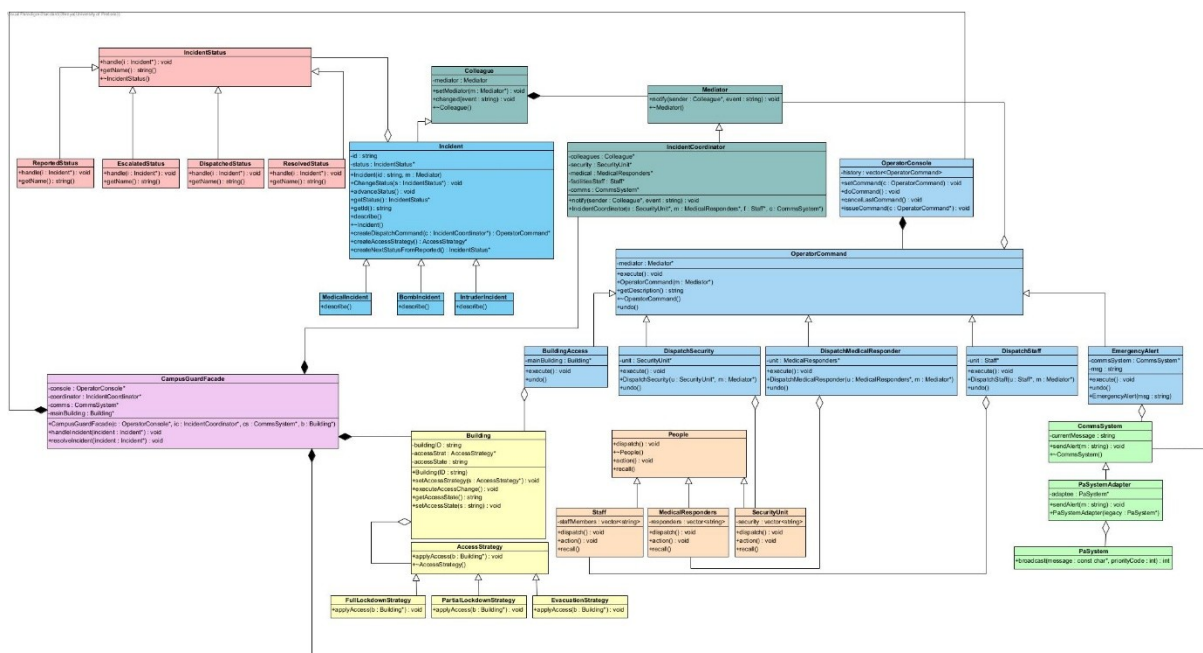
Team:

- u25443705
- u25108582
- u25245962

System Description

Our Campus Guard system starts with the client reporting an incident. Once an Incident is reported, depending on the type of incident it is, the status of the incident will become either dispatched or escalated, and then eventually resolved. If the state becomes Dispatched then depending on what unit (medical response, security or staff) needs to be dispatched, it will be dispatched and the respective unit will then do what they need to do. The access properties of the building may also change if the Incident is a bomb threat or an intruder. For a bomb the building will go on partial lockdown, for an intruder, full lock down. A Pa announcement will be sent out for all incidents.

UML Diagram



GoF Participants

Command

- Command – OperatorCommand
- ConcreteCommands – BuildingAccess / DispatchSecurity / DispatchMedicalResponder / DispatchStaff / EmergencyAlert
- Recievers – Staff, MedicalResponders, SecurityUnit
- Invoker – OperatorConsole

Mediator

- Mediator
- Colleague
- ConcreteMediator – IncidentMediator
- ConcreteColleague – Incident

Adaptor

- Adaptee – PaSystem
- Target – CommsSystem
- Adapter – PaSystemAdapter

Facade

- Facade – CampusGuardFacade

Added Design Patterns

State

- State – IncidentStatus

- ConcreteStates – ReportedStatus / EscalatedStatus / DispatchedStatus / ResolvedStatus
- Context – Incident

Strategy

- Strategy – AccessStrategy
- ConcreteStrategies – FullLockdownStrategy / PartialLockdownStrategy / EvacuationStrategy
- Context – Building

Justification

Incidents will have a process in which they are handled. State pattern allows for 'phases' (ReportedStatus, EscalatedStatus, DispatchedStatus, ResolvedStatus) and the operations to differ based on what phase it is in.

Better than having a nested if/else or a switch statement, allows for new concretestates being added without modifying the existing Incident class.

Different scenarios will need different access types for access to a Building. How each of these are handled can easily be described by each concreteStrategy.

Strategy pattern can be dynamically changed at runtime, and new safety protocols can be easily added/implemented at a later time.

Relationships and Ownership decisions:

Most relationship and ownership decisions were made based off how the certain design patterns work. The façade has a composite relationship with OperatorConsole, the CommsSystem, Building and IncidentCoordinator. The OperatorConsole owns an OperatorCommand which has 4 subclasses that inherit from it (concrete commands) which each hold objects that correspond with that command. Incident has 3 subclasses that inherit from it and it holds

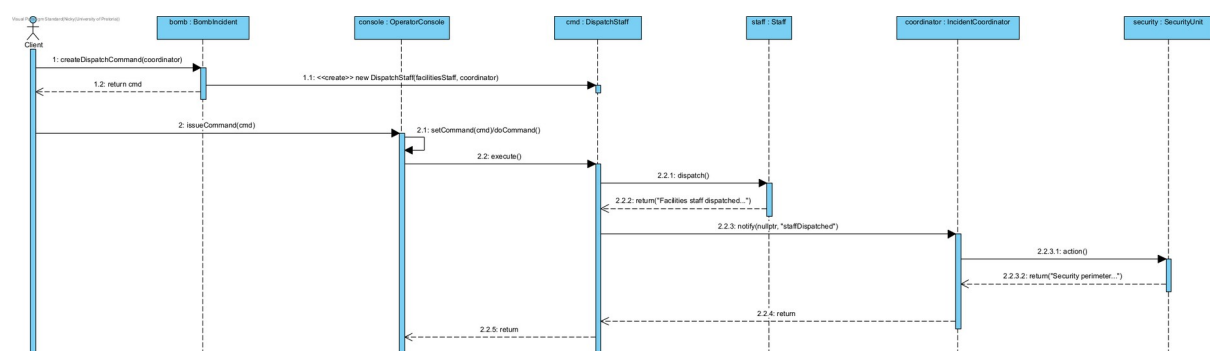
a pointer to an IncidentStatus object. Incident inherits from colleague, and every colleague owns a mediator.

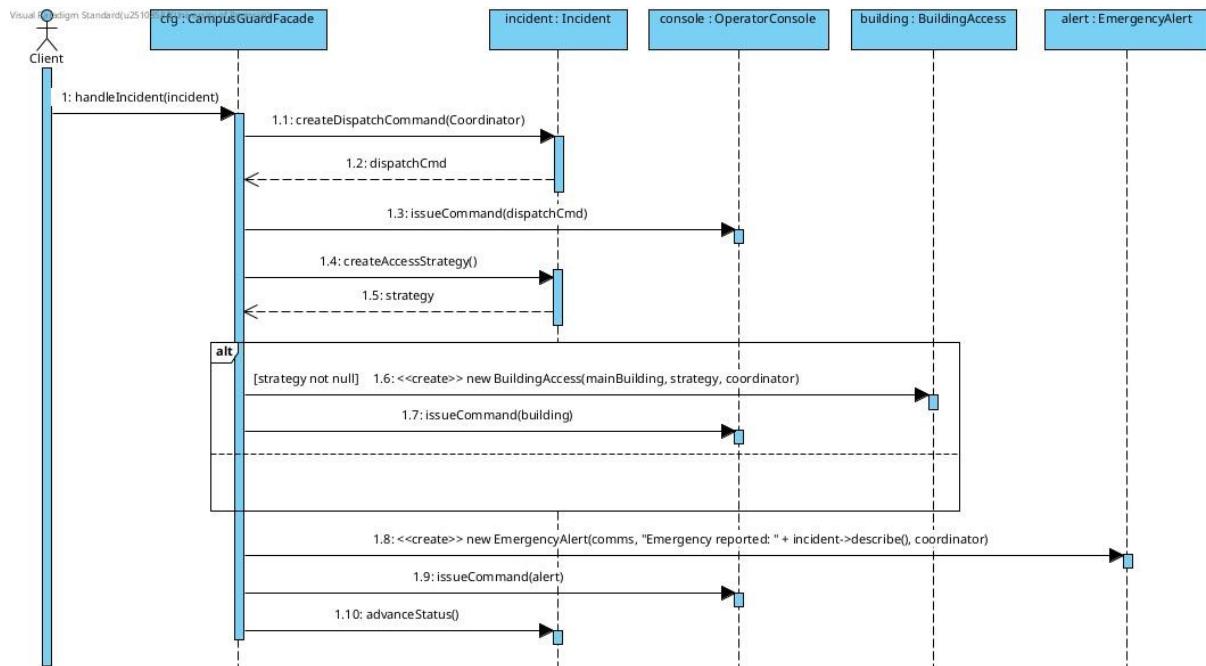
Design Pattern Interactions

The campusGuardFacade is connected to the operatorConsole which controls building access, dispatching units and the PA system, The concrete Commands each hold a pointer to an object that they perform actions on like changing building access permissions, dispatching units and sending out announcements. The state of an incident is what allows for certain steps to take place like FullLockDown a building or to dispatching . The IncidentCoordinator (concrete mediator) holds references to all the colleagues (the units). These subclasses notify the mediator. The concrete strategies override the applyAccess function to dynamically change the access permissions of a building object. The concrete States all have handle functions to control the state transitions (advanceStatus and changeStatus). The current state is what controls which operations can be executed.

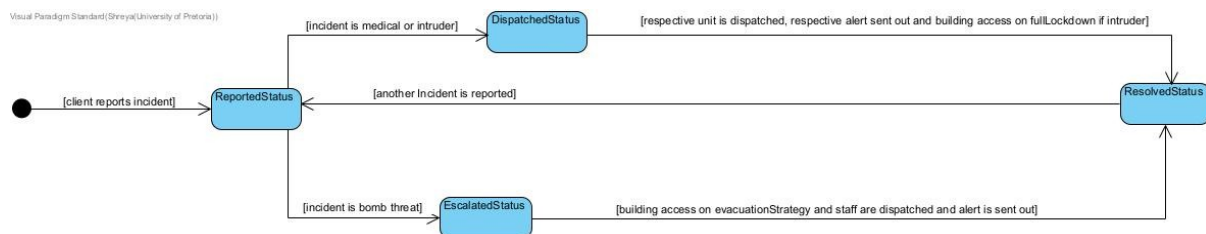
Diagrams

Sequence Diagrams:





Behavioural Diagram:



GDB and Valgrind

```

(gdb) s
OperatorConsole::~OperatorConsole (this=0x7ffca998ebf0, __in_chrg=<optimized out>) at src/OperatorConsole.cpp:41
41   for (size_t i = 0; i < history.size(); ++i) {
(gdb) s
std::vector<OperatorCommand*, std::allocator<OperatorCommand*> >::size (this=0x7ffca998ebf0) at /usr/include/c++/13/bits/stl_vector.h:993
993   { return size_type(this->_M_impl._M_finish - this->_M_impl._M_start); }
(gdb) s
OperatorConsole::~OperatorConsole (this=0x7ffca998ebf0, __in_chrg=<optimized out>) at src/OperatorConsole.cpp:42
42   delete history[i];
(gdb) s
std::vector<OperatorCommand*, std::allocator<OperatorCommand*> >::operator[] (this=0x7ffca998ebf0, __n=4) at /usr/include/c++/13/bits/stl_vector.h:1129
1129   return *(this->_M_impl._M_start + __n);
(gdb) s
1130   }
(gdb) s
Program received signal SIGSEGV, Segmentation fault.
  
```

Ownership of strategy was transferred to BuildingAccess. Deleting strategy then deleting commSystem causes a seg fault as OperatorConsole calls delete history[i] which deletes BuildingAccess (since BuildingAccess owns strategy, it will try to delete it but it was already deleted)

```
==65== Memcheck, a memory error detector
==65== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==65== Using Valgrind-3.14.0 and LibVEX; rerun with -h for copyright info
==65== Command: ./campusguard
==65==
==65==
==65== HEAP SUMMARY:
==65==     in use at exit: 0 bytes in 0 blocks
==65==   total heap usage: 115 allocs, 115 frees, 80,509 bytes allocated
==65==
==65== All heap blocks were freed -- no leaks are possible
==65==
==65== For counts of detected and suppressed errors, rerun with: -v
==65== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

GitHub

<https://github.com/nick-on-fleek/COS214-Prac5>